

Experiment 10: Heuristic Search

CSA1717 – Artificial Intelligence

Name: Hemalakshmi H

Reg.No.: 192571049

Aim:

To implement **Heuristic Search** using the **Greedy Best-First Search algorithm**, which selects the next node to explore based on the heuristic value estimating the distance to the goal, thereby improving search efficiency.

Algorithm:

Greedy Best-First Search Algorithm

1. Start with the initial node and place it in a priority queue.
2. Assign each node a heuristic value $h(n)$ — an estimate of the cost to reach the goal.
3. Select the node with the **lowest heuristic value** from the queue.
4. Expand that node and generate its successors.
5. For each successor:
 - If it has not been visited, add it to the queue with its heuristic value.
6. Repeat steps 3–5 until the goal node is selected.
7. Return the path from start to goal.

Source Code:

```
import heapq

def greedy_best_first_search(graph, start, goal, heuristic):

    # Priority queue based on heuristic value

    frontier = []

    heapq.heappush(frontier, (heuristic[start], [start]))

    visited = set()

    while frontier:

        h_val, path = heapq.heappop(frontier)
```

```

    node = path[-1]
    if node == goal:
        return path
    if node not in visited:
        visited.add(node)
        for neighbor in graph.get(node, []):
            if neighbor not in visited:
                new_path = path + [neighbor]
                heapq.heappush(frontier, (heuristic[neighbor], new_path))
    return None

# Example graph
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': ['G'],
    'F': [],
    'G': []
}

# Heuristic values (estimated distance to goal)
heuristic = {
    'A': 7,
    'B': 6,
    'C': 4,
    'D': 5,
    'E': 2,
    'F': 3,
    'G': 0
}

```

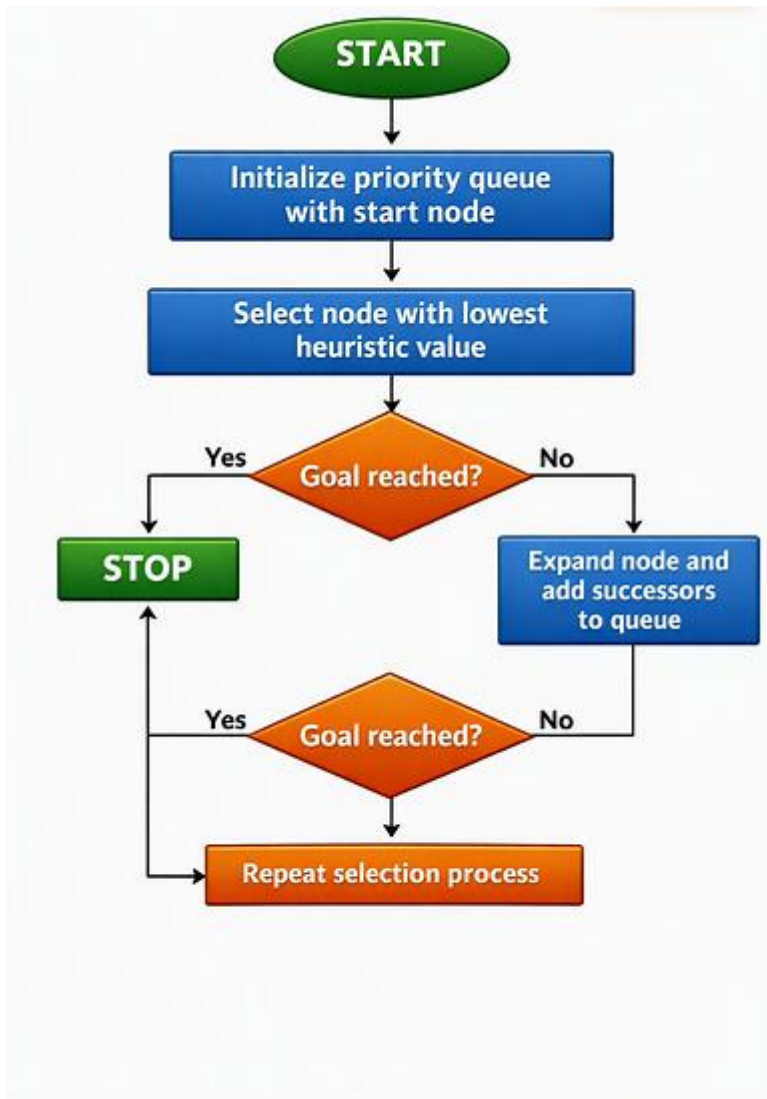
```

}

path = greedy_best_first_search(graph, 'A', 'G', heuristic)
print("Path found using Heuristic Search:", path)

```

Flowchart:



Output:

```

>>> |
>>> | == RESTART: C:/Users/hemal/OneDrive/Documents/SIMATS/AI/AI_PY/Heuristic_AI.py ==
>>> | Path found using Heuristic Search: ['A', 'B', 'E', 'G']
>>> |

```

Result:

The **Heuristic Search** algorithm successfully finds the goal node **G** starting from **A**, guided by heuristic values that estimate proximity to the goal. The final path obtained is:

$A \rightarrow B \rightarrow E \rightarrow G$

This confirms that heuristic search efficiently directs exploration toward the goal, reducing unnecessary node expansions compared to uninformed search methods.